

파이썬을 이용한 알기쉬운 수치해석

김시조

국립경국대학교: (구)국립안동대학교

February 17, 2026

머릿말

Note. 참고 URL

https://github.com/seejokim1/NA_with_python

21세기는 기술 혁신과 지식의 융합이 핵심 동력으로 작용하는 시대다. 특히 4차 산업혁명과 인공지능 기술의 발전은 전통적인 공학 분야에 새로운 도전과 기회를 제공하고 있다. 이러한 환경에서 수치해석은 공학 문제를 해결하는 핵심 도구로, 다양한 데이터 기반 분석과 최적화 문제 해결에 활용되고 있다.

현재 수치해석의 중요성은 더욱 강조되고 있으며, 특히 자율주행 자동차의 수치해석에서 가장 중요한 것은 정확도와 실시간 처리이다. 자율주행 차량은 다양한 센서 데이터를 실시간으로 처리하며, 도로 상황을 정확하게 예측하고 반응해야 하기 때문이다. 이러한 기술은 수치해석을 통해 더욱 정교해지고, 안전한 자율주행을 가능하게 한다.

또한 AI와 수치해석은 깊은 관계를 맺고 있으며, AI의 발전이 수치해석 방법론에 많은 영향을 미쳤고, 반대로 수치해석 기술이 AI의 성능을 향상시키는 데 중요한 역할을 해왔다. 수치해석은 실세계의 문제를 수학적으로 모델링하고 해결하기 위한 다양한 알고리즘과 방법을 제공하며, AI는 이러한 수학적 모델을 데이터 기반으로 학습하고 최적화하는 능력을 갖추고 있다. 이 둘의 결합은 현대 과학과 공학의 중요한 연구 분야로 자리 잡았다.

예를 들어, 2024년 노벨 물리학상은 머신러닝과 AI 기술이 물리학 문제 해결에 중요한 기여를 한 연구자에게 수여되었다. 이는 머신러닝이 복잡한 물리적 시스템을 시뮬레이션하거나 예측하는 데 사용되고 있음을 보여준다. 또한 알파고에 이어 알파폴드와 같은 AI 기술이 노벨 화학상을 수상하며, 수치해석 기법을 바탕으로 AI 모델이 실세계 문제를 해결하는 데 어떻게 활용될 수 있는지를 보여주었다.

이 책은 파이썬 프로그래밍 언어를 활용하여 수치해석의 기초부터 심화 내용까지 단계적으로 학습할 수 있도록 구성되었다. 첫 장에서는 파이썬의 기본 문법과 데이터 처리에 필수적인 라이브러리(예: NumPy, Matplotlib)를 다루며, 프로그래밍과 수치해석의 연결고리를 제공한다. 이어지는 장에서는 비선형 방정식 근사법, 선형 연립방정식 해법, 수치미분 및 수치적분, 보간법, 수치 회귀 등 전통적인 수치해석의 핵심 주제들을 체계적으로 소개한다.

특히 후반부에서는 상미분 방정식과 편미분 방정식의 수치적 해법을 다루며, 유한차분법(Finite Difference Method)과 유한요소법(Finite Element Method)의 응용을 통해 실질적인 문제 해결 방법을 제시한다. 이와 더불어 Python을 활용하여 다양한 수치해석 문제를 직접 해결해보는 연습문제를 포함해 독자가 이론과 실습을 병행하며 학습할 수 있도록 하였다.

김시조

Contents

1	파이썬의 이해	1
2	비선형 방정식 구하기	2
3	선형연립방정식 수치해법	3
4	수치미분 (Numerical Differentiation)	4
5	수치적분 (Numerical Integration)	5
6	수치 보간 (Interpolation)	6
7	수치 회귀 (Numerical Regression)	7
8	비선형연립방정식 수치해법	8
8.1	Newton-Raphson Method	8
8.1.1	비선형 연립방정식의 뉴턴-랩슨법 (Newton-Raphson Method)	8
8.2	비선형연립방정식 수치해법	10
8.2.1	8.2.1 원과 직선이 만나는 점을 찾는 연립 비선형방정식	10
8.2.2	원과 직선이 만나는 점을 찾는 연립 비선형방정식 (코드 구현)	11
8.3	연습문제	12

Chapter 1

파이썬의 이해

Chapter 2

비선형 방정식 구하기

Chapter 3

선형연립방정식 수치해법

Chapter 4

수치미분 (Numerical Differentiation)

Chapter 5

수치적분 (Numerical Integration)

Chapter 6

수치 보간 (Interpolation)

Chapter 7

수치 회귀 (Numerical Regression)

Chapter 8

비선형연립방정식 수치해법

8.1 Newton-Raphson Method

8.1.1 비선형 연립방정식의 뉴턴-랩슨법 (Newton-Raphson Method)

비선형 연립방정식

$$\mathbf{F}(\mathbf{x}) = \mathbf{0}$$

에 대해,

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - J^{-1}(\mathbf{x}^{(k)})\mathbf{F}(\mathbf{x}^{(k)})$$

여기서 J 는 Jacobian 행렬이다.

Jacobian:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

1. 문제 정의 다음과 같은 비선형 연립방정식을 생각하자.

$$(x)^2 + (y)^2 = 4, \quad y = 2x + 1$$

이를 벡터 함수 형태로 쓰면

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(x, y) \\ f_2(x, y) \end{bmatrix} = \begin{bmatrix} x^2 + y^2 - 4 \\ y - 2x - 1 \end{bmatrix} = \mathbf{0}$$

—

2. 일반적인 n 변수 비선형 시스템

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(x_1, \dots, x_n) \\ f_2(x_1, \dots, x_n) \\ \vdots \\ f_n(x_1, \dots, x_n) \end{bmatrix} = \mathbf{0}$$

—

3. Taylor 전개 (1차 근사)

$$\mathbf{f}(\mathbf{x} + \delta\mathbf{x}) \approx \mathbf{f}(\mathbf{x}) + J(\mathbf{x}) \delta\mathbf{x}$$

여기서 J 는 Jacobian 행렬:

$$J_{ij} = \frac{\partial f_i}{\partial x_j}$$

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \dots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}$$

4. 뉴턴 보정식 해 $\mathbf{x}$ 에서

$$J(\mathbf{x}) \delta\mathbf{x} = -\mathbf{f}(\mathbf{x})$$

이를 풀어 보정값을 구한다:

$$\delta\mathbf{x} = -J^{-1}(\mathbf{x})\mathbf{f}(\mathbf{x})$$

5. 반복 갱신식

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \delta\mathbf{x}_k$$

6. 수렴 조건

- 초기값 $\mathbf{x}_0$ 이 해 근처에 있어야 함
- Jacobian이 가역(역행렬 존재)이어야 함
- 함수가 충분히 매끄러워야 함
- $\|\delta\mathbf{x}\|$ 가 충분히 작아질 때까지 반복

7. 알고리즘 요약

1. 초기값 $\mathbf{x}_0$ 설정
2. $J(\mathbf{x}_k)\delta\mathbf{x}_k = -\mathbf{f}(\mathbf{x}_k)$ 계산
3. $\mathbf{x}_{k+1} = \mathbf{x}_k + \delta\mathbf{x}_k$
4. 수렴 여부 확인 후 반복

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter08/section_08_01_01_nonlinear_system_newton_raphson.py

8.2 비선형연립방정식 수치해법

8.2.1 8.2.1 원과 직선이 만나는 점을 찾는 연립 비선형방정식

1. 문제 설정

원과 직선의 교점을 뉴턴-랩슨 방법으로 구한다.

$$(x)^2 + (y)^2 = 4 \quad (8.1)$$

$$y = 2x + 1 \quad (8.2)$$

두 함수로 표현하면

$$f_1(x, y) = x^2 + y^2 - 4 \quad (8.3)$$

$$f_2(x, y) = y - 2x - 1 \quad (8.4)$$

벡터 함수 형태:

$$\mathbf{F}(x, y) = \begin{bmatrix} f_1(x, y) \\ f_2(x, y) \end{bmatrix} = \begin{bmatrix} x^2 + y^2 - 4 \\ y - 2x - 1 \end{bmatrix}$$

2. Jacobian 행렬

$$J(x, y) = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x & 2y \\ -2 & 1 \end{bmatrix}$$

3. 뉴턴 반복식

선형화:

$$J(x_k) \delta \mathbf{x} = -\mathbf{F}(x_k)$$

갱신식:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \delta \mathbf{x}$$

4. 수렴 조건

$$\|\delta \mathbf{x}\| < \varepsilon$$

(보통 maximum norm, 1-norm, 2-norm 사용)

5. 계산 절차

(a) 초기값 $\mathbf{x}_0$ 설정

(b) $\mathbf{F}(\mathbf{x}_k)$, $J(\mathbf{x}_k)$ 계산

(c) 선형시스템 풀이:

$$J(\mathbf{x}_k) \delta \mathbf{x} = -\mathbf{F}(\mathbf{x}_k)$$

(d) 갱신:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \delta \mathbf{x}$$

(e) $\|\delta \mathbf{x}\| < \varepsilon$ 까지 반복

8.2.2 원과 직선이 만나는 점을 찾는 연립 비선형방정식 (코드 구현)

1. 문제 설정

$$x^2 + y^2 = 1 \quad (8.5)$$

$$y = 2x + 1 \quad (8.6)$$

벡터 함수:

$$\mathbf{F}(x, y) = \begin{bmatrix} x^2 + y^2 - 1 \\ y - 2x - 1 \end{bmatrix}$$

2. Jacobian 행렬

$$J(x, y) = \begin{bmatrix} 2x & 2y \\ -2 & 1 \end{bmatrix}$$

3. Python Jacobian 함수

```
def jx1(x2):
    j11 = 2.0 * x2[0]    # ∂f1/∂x
    j12 = 2.0 * x2[1]    # ∂f1/∂y
    j21 = -2.0           # ∂f2/∂x
    j22 = 1.0             # ∂f2/∂y
    return array([[j11, j12],
                  [j21, j22]])
```

4. Newton-Raphson 반복

선형 시스템:

$$J(\mathbf{x}_k)\delta\mathbf{x} = -\mathbf{F}(\mathbf{x}_k)$$

갱신식:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \delta\mathbf{x}$$

5. 알고리즘 개요

- (a) 초기값 설정
- (b) $\mathbf{F}(\mathbf{x}_k)$ 계산
- (c) $J(\mathbf{x}_k)$ 계산
- (d) 선형 시스템 풀이
- (e) $\mathbf{x}_{k+1} = \mathbf{x}_k + \delta\mathbf{x}$
- (f) 수렴 조건 만족 시 종료

실습예제: https://github.com/seejokim1/NA_with_python/blob/main/chapter08/section_08_02_02_circle_line_nonlinear_system.py

8.3 연습문제